

基于分阶段动力学与控制参数化的两级运载火箭入轨轨迹优化

摘 要

运载火箭入轨轨迹设计直接影响有效载荷投送精度与推进剂利用效率。针对两级液体运载火箭从起飞、级间滑行到二子级入轨的连续过程,本文分别建立分阶段变质量动力学仿真模型、三参数非线性打靶模型、燃料最优控制参数化模型和可节流燃料最优模型,用于完成基准轨迹复现、圆轨道参数求解以及二子级推力方向与大小优化。

首先统一质量、推力、比冲、角度和时间单位,在地心惯性极坐标系内以地心距、射程角、径向速度、切向速度和总质量描述状态;将赤道发射点的地球自转速度计入初始条件,并以旋转大气相对速度计算一子级阻力。级间分离通过质量跃变处理,三个飞行阶段均采用八阶 **Dormand-Prince 自适应 Runge-Kutta 积分法 (DOP853)** 和 **燃尽事件定位**。

问题一在题定控制下完成全程仿真。一子级于 **149.061s** 燃尽,分离高度 **109.265km**,接着无动力滑行 60s,到达高度 **210.896km**,二子级在 **634.670s** 燃尽时达到 **433.527km**,径向速度为 **-102.023m/s**,切向速度为 **8520.568m/s**,飞行路径角为 **-0.686°**。该轨迹速度高于 400km 圆轨道速度,不能直接满足目标入轨条件。

问题二以滑行时间、二子级俯仰角变化率和二子级工作时间为三个待定量,将 400km 高度、零径向速度和圆轨道切向速度构成三维残差,利用**基于 Powell 混合算法的非线性方程求解器 fsolve** 进行 27 组多初值打靶。15 组初值收敛并去重为唯一根:滑行 **103.632s**、俯仰角变化率 **-0.047°/s**、工作 **398.049s**,关机后剩余推进剂 **4014.715kg**。

问题三将二子级推力—速度夹角离散为分段线性节点,以工作时间最短等价表征推进剂消耗最少,采用**序列最小二乘规划 (Sequential Least Squares Programming, SLSQP)** 直接打靶。控制节点由 9、17、33 逐步加密至 65 个,求解获得的剩余推进剂由 **4509.878kg** 稳定至 **4509.941kg**,其中 65 节点方案的滑行时间为 **27.384s**,工作时间为 **394.649s**,较问题二少消耗推进剂约 **495.226kg**。

问题四引入 60%~100% 的连续节流比,以节流积分定义等效满推力时间,并同时优化滑行时间、推力—速度夹角节点、节流节点和燃料指标。三组独立初值及 9、17、33 节点加密均收敛到全程满推力候选,33 节点剩余推进剂 **4509.933kg**,与问题三同精度解一致。将 DOP853 最大步长依次取 0.1、0.25、0.5、1、2 和 5s,终端缩放残差保持在 **$1.19 \times 10^{-10} \sim 1.90 \times 10^{-10}$** ,表明数值结果稳定。

关键词: 分阶段动力学; 非线性打靶; SLSQP; 推力节流

1 问题背景与重述

1.1 问题背景

运载火箭从地面起飞到进入预定轨道，需要在有限推进剂条件下同时完成高度提升、水平加速和轨道圆化。飞行过程中，中心引力、大气阻力、地球自转、质量持续下降和级间分离共同改变火箭状态；滑行时长与二子级推力方向还会影响重力损失和推进剂利用率。因此，建立统一的分阶段动力学模型并对末级控制进行优化，是评价入轨能力和设计节能轨迹的基础。

1.2 问题重述

某两级液体运载火箭从赤道发射场起飞，目标是将 10t 有效载荷送入高度 400km 的近地圆轨道。飞行过程依次包括一子级垂直上升与程序转弯、一二子级分离后的无动力滑行以及二子级点火入轨。题目给定两级质量、推进剂、推力、比冲及气动参数，要求建立动力学模型并完成四项任务：复现基准控制下的全程轨迹；调整滑行时间和二子级线性俯仰程序实现圆轨道入轨；进一步优化滑行时间和俯仰角程序以减少二子级耗油；在二子级可于 60%~100% 额定推力间连续节流时重新求解最省油控制策略。

2 问题分析

2.1 问题一分析

问题一的控制律和各阶段持续条件均已给定，核心是构造能够统一描述推力、中心引力、旋转大气阻力和变质量效应的分阶段动力学系统。连续飞行由常微分方程刻画，一子级燃尽与二子级燃尽由推进剂耗尽事件定位，级间分离则表现为质量的瞬时跃变。

考虑到起飞瞬间速度为零，直接把速度大小和飞行路径角作为独立状态会出现路径角初值不稳定的问题，因此采用径向速度与切向速度作为基本状态，再由二者计算速度和路径角。求解结果除给出四条状态曲线外，还需检查一、二子级燃尽时刻、分离质量跃变以及终端状态是否满足目标圆轨道条件。

2.2 问题二分析

问题二保留问题一的一子级轨迹，把滑行时间、二子级俯仰角变化率和二子级工作时间作为未知量。目标圆轨道提供高度、径向速度和切向速度三项独立条件，因此可建立三元非线性打靶方程，并通过多初值求根检验解的稳定性和唯一性。

三个待定参数对终端状态的影响具有明显耦合：滑行时间改变点火高度和速度方向，俯仰角变化率决定推力在径向与切向上的分配，工作时间控制总速度增量和耗油

量。求解时需要对三项终端残差进行尺度归一化，并排除工作时间超过燃尽极限或剩余推进剂为负的非物理解。

2.3 问题三分析

问题三中的二子级推力恒定，推进剂质量流率为常数，最小耗油量与最短工作时间等价。俯仰角程序属于连续函数，采用控制参数化把推力—速度夹角离散为有限节点，在每个候选控制下正向积分状态方程，并用圆轨道终端条件约束非线性规划。网格加密用于检验控制离散误差。

该问题的决策对象由三个标量参数扩展为一条连续控制函数，属于有限时间燃料最优控制问题。将推力方向写成速度方向与相对夹角之和，可以直接施加夹角边界，并避免分别限制俯仰角和路径角。节点数增加会提高控制自由度，同时也扩大非线性规划规模，因此采用由粗到细的网格和热启动策略。

2.4 问题四分析

问题四增加节流比后，质量流率不再恒定，不能继续以工作时间直接衡量燃料。应以节流比的时间积分构造燃料目标，并同步优化夹角和节流节点。由于扩大控制集合并不必然严格改善目标值，需要通过多初值、网格加密和独立高精度复算判断节流是否真正产生省油收益。

节流降低会减小单位时间耗油率，但也会延长有动力飞行并增加重力作用时间，因此最优节流不应仅凭直观指定。模型必须允许节流节点在 60%~100% 区间自由变化，再由终端圆轨道约束和燃料目标共同决定其取值；最后还应将问题四与问题三在相同节点数下比较，以区分节流收益与离散误差。

3 模型的假设

模型假设主要包括：

- 1、地球为均匀球体，重力采用中心引力模型，火箭在赤道平面内运动。
- 2、火箭视为变质量质点，忽略姿态动力学、结构振动、推力建立过程及级间分离冲量。
- 3、一子级采用题给指数大气密度和常阻力系数；大气随地球刚性旋转，二子级阶段忽略气动阻力。
- 4、发动机比冲与额定推力在各自工作阶段保持不变，关机前不考虑二次点火。
- 5、问题三、四中，二子级推力—速度夹角满足 $|\alpha| \leq 15^\circ$ ，作为控制可实现性的附加约束；不另设角速度与节流变化率约束。
- 6、400 km 圆轨道以终端高度 400 km、径向速度为 0 和切向速度等于当地圆轨道速度表征。

4 符号说明

4.1 常量声明

本题目求解过程中，使用到的常量定义如表 1所示。

表 1 常量说明

符号	常量值	单位	含义
R_E	6.371×10^6	m	地球半径
Ω_E	7.292115×10^{-5}	$\text{rad} \cdot \text{s}^{-1}$	地球自转角速度
μ	$3.986004418 \times 10^{14}$	$\text{m}^3 \cdot \text{s}^{-2}$	地球引力参数
g_0	9.80665	$\text{m} \cdot \text{s}^{-2}$	标准重力加速度
I_{spi}	题目给定	s	第 i 级发动机比冲
$F_{\max i}$	题目给定	N	第 i 级发动机额定最大推力

4.2 变量声明

本题目求解过程中，使用到的主要变量及其符号定义见表 2。

表 2 变量说明

符号	单位	含义
r	m	地心距
h	m	火箭高度
v_t	$\text{m} \cdot \text{s}^{-1}$	切向速度
v_r	$\text{m} \cdot \text{s}^{-1}$	径向速度
θ	rad	惯性极角
γ	rad	飞行路径角
ψ	rad	推力俯仰角
m	kg	火箭总质量
T	N	发动机推力
u	无量纲	二子级节流比
D_r	N	阻力的径向分量
D_t	N	阻力的切向分量
ρ	$\text{kg} \cdot \text{m}^{-3}$	大气密度
τ	s	滑行时间
t_{c2}	s	二子级工作时间
k_2	$^\circ \cdot \text{s}^{-1}$	二子级俯仰角变化率
q	s	等效满推力时间

5 模型的建立与求解

5.1 问题一模型建立及求解

5.1.1 状态变量与环境模型

在地心惯性极坐标系内选取状态向量 $\mathbf{x} = [r, \theta, v_r, v_t, m]^T$ 。速度大小与飞行路径角分别为：

$$\begin{aligned} v &= \sqrt{v_r^2 + v_t^2}, \\ \gamma &= \text{atan2}(v_r, v_t). \end{aligned} \quad (1)$$

赤道发射点在惯性系中具有向东切向速度，则初始状态可写为：

$$\mathbf{x} = [R_E, 0, 0, v_0, M_0]^T \quad (2)$$

其中， $v_0 = \Omega_E R_E$ 。

采用径向一切向速度状态后，起飞时无需人为指定飞行路径角；当速度从零逐渐增大时，可由 $\text{atan2}(v_r, v_t)$ 连续恢复路径角。火箭在当地基底上的矢量受力关系为

$$m\dot{\mathbf{v}} = T(\sin\psi\mathbf{e}_r + \cos\psi\mathbf{e}_t) + D_r\mathbf{e}_r + D_t\mathbf{e}_t - \frac{\mu m}{r^2}\mathbf{e}_r \quad (3)$$

大气相对速度的切向分量为 $v_{rel,t} = v_t - \Omega_E r$ ，径向分量仍为 v_r 。由题给指数密度模型得到一子级阻力分量：

$$\begin{aligned} V_{rel} &= \sqrt{v_r^2 + v_{rel,t}^2}, \\ \rho(h) &= \rho_0 e^{-\frac{h}{h_0}}, \\ D_r &= -\frac{1}{2}\rho C_D S V_{rel} v_r, \\ D_t &= -\frac{1}{2}\rho C_D S V_{rel} (v_t - \Omega_E r). \end{aligned} \quad (4)$$

其中 $S = 12.5$ 为题目给定的一子级飞行气动阻力参考面积， $C_D = 0.3$ 为阻力系数。

5.1.2 分阶段变质量动力学

推力俯仰角 ψ 以当地水平面为零方向。将推力、阻力与中心引力分别投影到径向和切向，可得统一动力学方程。

$$\begin{cases} \dot{r} = v_r \\ \dot{\theta} = v_t/r, \\ \dot{v}_r = v_r^2/r - \mu/r^2 + (T\sin\psi + D_r)/m, \\ \dot{v}_t = -v_r v_t/r + (T\cos\psi + D_t)/m, \\ \dot{m} = -T/(I_{sp} g_0). \end{cases} \quad (5)$$

其中 v_t^2/r 与 $-v_r v_t/r$ 来自极坐标基底随位置转动产生的几何项。飞行阶段运载火箭的质量变化由发动机消耗燃料产生。当发动机关闭时, $T = 0$, 运载火箭质量不再发生变化。

一子级仅在前 10s 保持 $\psi_1 = 90^\circ$, 此后按 $0.4^\circ/s$ 线性减小。燃尽条件为 $m = M_0 - m_{p1}$ 。随后瞬时抛离 40t 结构质量。滑行阶段取 $T = 0$ 且忽略阻力; 60s 后二子级全推力点火并令 $\psi_2 = \gamma$, 直至将燃料消耗完毕。由题意可列出运载火箭飞行阶段的判定函数:

$$\begin{cases} g_1(\mathbf{x}) = m - (M_0 - m_{p1}) \\ g_2(\mathbf{x}) = m - (m_{s2} + m_{payload}). \end{cases} \quad (6)$$

其中事件 $g_1(\mathbf{x}) = 0$ 时, 一子级推进剂耗尽、发动机关机, 同理事件 $g_2(\mathbf{x}) = 0$ 表示二子级耗尽推进剂。位置和速度在分离瞬间连续, 仅质量发生跃变, 因而整个模型属于连续动力学与离散事件组合的混杂系统。各飞行阶段运载火箭的受力、速度与推力方向关系如图 1 所示。

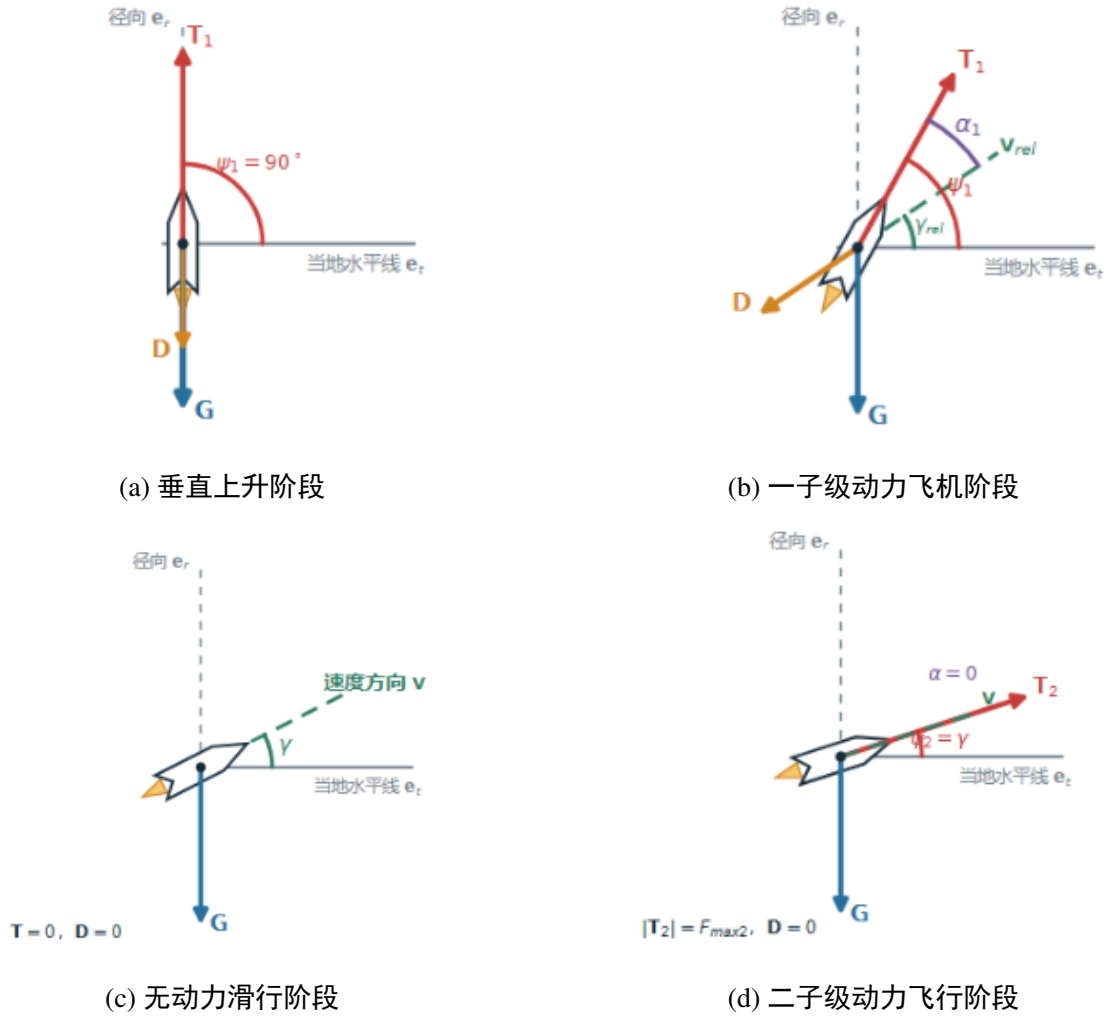


图 1 各飞行阶段的受力、速度与推力方向关系

5.1.3 DOP853 数值求解

对于运载火箭飞行过程中的各连续阶段，使用 DOP853 显式高阶 Runge-Kutta 方法。对常微分方程 $\dot{\mathbf{x}} = \mathbf{f}(t, \mathbf{x})$ ，单步内部阶段量与更新式写为：

$$\begin{cases} \mathbf{k}_i = \mathbf{f}(t_n + c_i h_n, \mathbf{x}_n + h_n \sum_{j=1}^{i-1} a_{ij} \mathbf{k}_j) \\ \mathbf{x}_{n+1} = \mathbf{x}_n + h_n \sum_{i=1}^s b_i \mathbf{k}_i \end{cases} \quad (7)$$

由嵌入低阶公式估计局部误差 e_n 并自适应更新步长。相对误差容限取 10^{-9} 、绝对误差容限取 10^{-12} ，燃尽事件用质量零点定位，从而避免以固定步长近似燃尽时刻。

$$h_{n+1} = h_n \text{clip} \left[f_{min}, f_{max}, f_s \left(\frac{tol}{e_n} \right)^{1/p+1} \right] \quad (8)$$

其中 $\text{clip}()$ 为阶段操作，将结果限定在 f_{min} 与 f_{max} 阈值之间。

则算法的整体流程为：积分一子级直到事件 $g_1(\mathbf{x}) = 0$ ，执行质量跃变；以分离后状态积分 60s 滑行阶段；再以二子级全推力积分到 $g_2(\mathbf{x}) = 0$ 。各段末状态作为下一段初状态，使位置、速度和时间轴连续衔接。

5.1.4 问题一结果

通过上述数值仿真，基准策略关键时刻与状态如表 3 所示：

表 3 基准策略关键时刻与状态

时刻阶段	t (s)	h (km)	v_r ($\text{m} \cdot \text{s}^{-1}$)	v_t ($\text{m} \cdot \text{s}^{-1}$)	γ ($^\circ$)	m (t)
一子级燃尽	149.06	109.26	1944.27	2616.99	36.61	120.00
分离后	149.06	109.26	1944.27	2616.99	36.61	80.00
滑行结束	209.06	210.90	1445.86	2576.58	29.30	80.00
二子级燃尽	634.67	433.53	-702.02	8520.57	-0.69	18.00

由2(a)可知，二子级燃料燃尽时高度已超过 400km，且火箭切向速度 8520.57 m/s 高于该高度附近的圆轨道速度。基准策略只能给出完整轨迹，不能使有效载荷准确进入题定圆轨道。

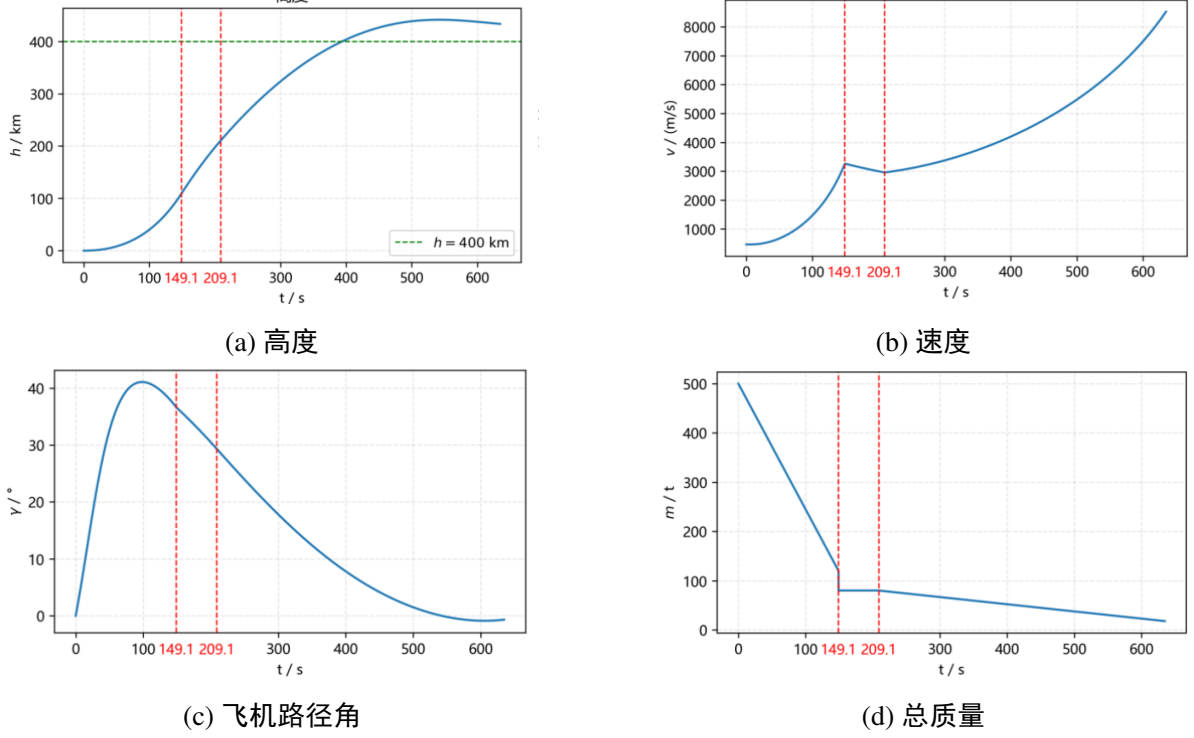


图 2 基准控制策略下全过程状态曲线

5.2 问题二模型建立及求解

5.2.1 三参数打靶模型

设一子级分离后的滑行时间为 τ ，二子级点火时刻的速度方向为 γ_{ign} 。二子级推力俯仰角以恒定速率 k_2 变化，二子级工作时间为 t_{c2} ，因此

$$\psi_2(s) = \gamma_{\text{ign}} + k_2 s, \quad 0 \leq s \leq t_{c2}. \quad (9)$$

目标圆轨道半径及其对应的圆轨道速度分别为

$$\begin{aligned} r_{\text{tar}} &= R_E + H, \\ v_c &= \sqrt{\frac{\mu}{r_{\text{tar}}}} \end{aligned} \quad (10)$$

从而定义待定参数向量与相应的三维打靶残差向量分别为

$$\begin{aligned} \mathbf{p} &= [\tau, k_2, t_{c2}]^T, \\ \mathbf{F}(\mathbf{p}) &= [r_f - r_{\text{tar}}, v_{rf}, v_{tf} - v_c]^T \end{aligned} \quad (11)$$

记滑行动力学的流映射为 Φ_c ，二子级动力学的流映射为 Φ_2 ，则从一子级分离状态 $\mathbf{x}_{\text{sep}}$ 到终端状态 $\mathbf{x}_f$ 的参数映射可表示为

$$\begin{aligned} \mathbf{x}_{\text{ign}} &= \Phi_c(\tau; \mathbf{x}_{\text{sep}}), \\ \mathbf{x}_f &= \Phi_2(t_{c2}; \mathbf{x}_{\text{ign}}, k_2). \end{aligned} \quad (12)$$

由于高度残差与速度残差的数量级不同，直接求根可能导致高度残差项在数值计算中占主导。因此，对残差向量进行尺度缩放：

$$\tilde{\mathbf{F}}(\mathbf{p}) = \left[\frac{r_f - r_{\text{tar}}}{1000}, \frac{v_{r,f}}{10}, \frac{v_{t,f} - v_c}{10} \right]^T \quad (13)$$

5.2.2 fsolve 求解方法

每给定一组 $\mathbf{p}$ ，均按“滑行—二子级有动力飞行”顺序调用 DOP853，得到终端残差。fsolve 采用 Powell 混合算法，在 Newton 步与信赖域梯度步之间调整。其局部线性模型可写为

$$\begin{aligned} \mathbf{d}_q &= \arg \min_{\|\mathbf{D}\mathbf{d}\| \leq \Delta_q} \|\mathbf{F}(\mathbf{p}_q) + \mathbf{J}_F(\mathbf{p}_q)\mathbf{d}\|, \\ \mathbf{p}_{q+1} &= \mathbf{p}_q + \mathbf{d}_q, \end{aligned} \quad (14)$$

其中， $\mathbf{D}$ 为变量尺度矩阵， Δ_q 为第 q 次迭代的信赖域半径， $\mathbf{J}_F$ 为残差函数 $\mathbf{F}$ 关于参数向量 $\mathbf{p}$ 的 Jacobian 矩阵。

当未显式提供解析 Jacobian 时，程序用差分近似各列敏感度

$$\mathbf{J}_{F,j}(\mathbf{p}_q) \approx \frac{\mathbf{F}(\mathbf{p}_q + \delta_j \mathbf{e}_j) - \mathbf{F}(\mathbf{p}_q)}{\delta_j}, \quad (15)$$

其中， δ_j 为第 j 个参数的差分步长， $\mathbf{e}_j$ 为第 j 个坐标方向上的单位向量。该 Jacobian 同时反映了滑行时间、俯仰角变化率和二子级工作时间对三个终端状态残差的耦合影响。

候选根需满足 $\tau \geq 0$ 、 $0 \leq t_{c2} \leq m_{p2} I_{sp2} g_0 / F_{max2}$ ，并以高精度积分重新计算终端残差。为降低非线性方程对初值的敏感性，对滑行时间、俯仰角变化率和工作时间分别取 3 档初值，形成 27 组组合；收敛解还需满足工作时间不超过二子级理论燃尽时间且终端残差通过高精度复算。

5.2.3 问题二结果

表 4 问题二最优打靶参数与终端状态

项目	数值	项目	数值
滑行时间 τ/s	103.631602	二子级工作时间 t_{c2}/s	398.048975
俯仰角变化率 $k_2/(\text{^\circ} \cdot \text{s}^{-1})$	-0.047365	总关机时刻 t_f/s	650.741657
终端高度 h_f/km	400.000	终端径向速度 $v_{r,f}/(\text{m} \cdot \text{s}^{-1})$	≈ 0
终端切向速度 $v_{t,f}/(\text{m} \cdot \text{s}^{-1})$	7672.599	剩余推进剂质量 $m_{p,f}/\text{kg}$	4014.72
多初值收敛数	15/27	去重后有效根数	1

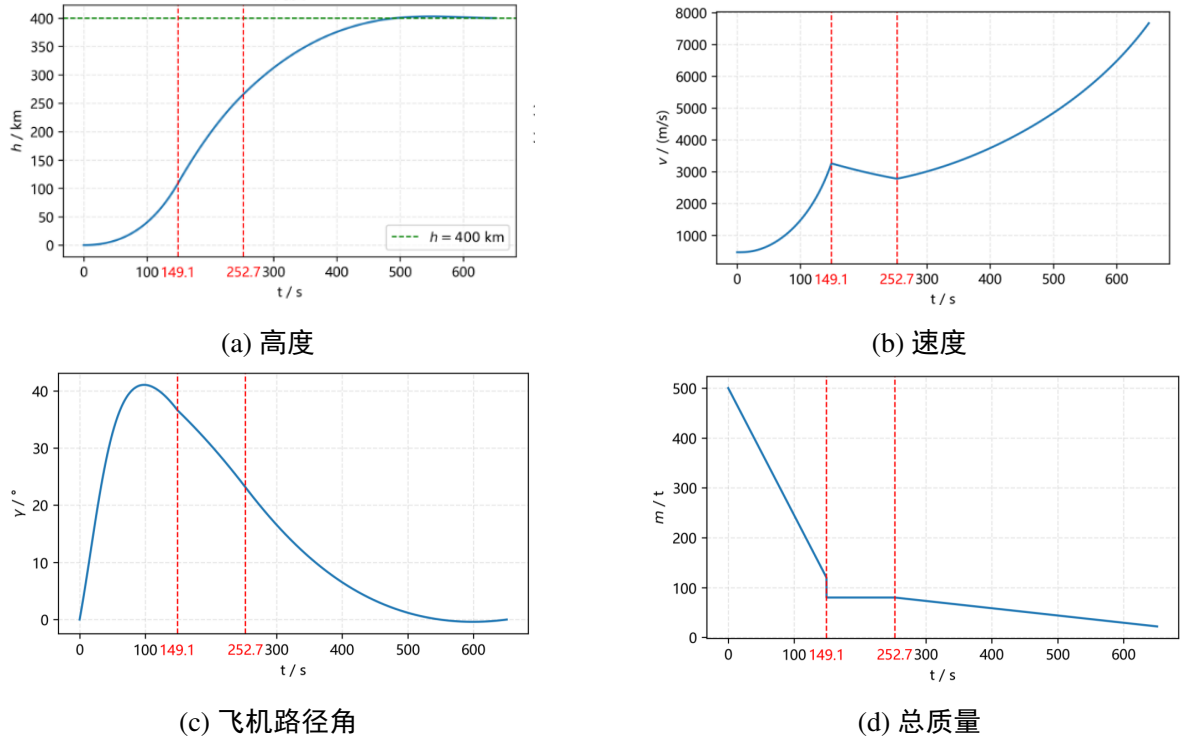


图 3 基准控制策略下全过程状态曲线

终端高度、径向速度与切向速度同时满足 400km 圆轨道条件,且二级级尚余 4014.72 kg 推进剂。多初值结果去重后仅得到一个可接受根,说明该三参数控制形式下求解结果具有较好的初值稳定性。

5.3 问题三模型建立及求解

5.3.1 燃料最优控制模型

二级级发动机保持额定推力 $F_{\max,2}$, 且推进剂质量流率恒定, 因此推进剂消耗量与二级级工作时间成正比, 即

$$m_{\text{fuel}} = \frac{F_{\max,2}}{I_{\text{sp},2}g_0}t_{c2}, \quad (16)$$

其中, $I_{\text{sp},2}$ 为二级级发动机比冲, g_0 为标准重力加速度。

定义推力方向与速度方向之间的夹角为

$$\alpha = \psi - \gamma, \quad (17)$$

并对其施加如下约束:

$$|\alpha(t)| \leq \alpha_{\max}, \quad \alpha_{\max} = 15^\circ. \quad (18)$$

该约束与大气层外运载火箭轨迹优化相关文献采用的攻角上限处于相同数量级^[4;5], 本文仅将其作为附加的控制可实现性条件。

由此，轨迹优化问题可以表示为

$$\begin{aligned} & \min_{\tau, \alpha(t), t_{c2}} t_{c2} \\ \text{s.t. } & \mathbf{F}(\mathbf{p}) = \mathbf{0}, \\ & |\alpha(t)| \leq \alpha_{\max}. \end{aligned} \quad (19)$$

由于二子级额定推力和发动机比冲均为常数，推进剂消耗量是工作时间 t_{c2} 的正比例函数。因此，最小化二子级工作时间与最小化推进剂消耗量完全等价。

目标圆轨道由终端半径、径向速度和切向速度三个条件共同确定：

$$\begin{cases} r(t_f) = R_E + H, \\ v_r(t_f) = 0, \\ v_t(t_f) = \sqrt{\frac{\mu}{R_E + H}}. \end{cases} \quad (20)$$

5.3.2 控制参数化直接打靶

在二子级局部时间区间 $(0, t_{c2})$ 上设置 K 个等距节点。节点间采用线性插值在相邻控制节点 t_j 和 t_{j+1} 之间，采用线性插值确定推力与速度之间的夹角：

$$\alpha(t) = \frac{t_{j+1} - t}{t_{j+1} - t_j} \alpha_j + \frac{t - t_j}{t_{j+1} - t_j} \alpha_{j+1}, \quad t_j \leq t \leq t_{j+1}. \quad (21)$$

对于包含 K 个控制节点的离散模型，定义决策向量为

$$\mathbf{z}_K = [\tau \quad \alpha_0 \quad \cdots \quad \alpha_{K-1} \quad t_{c2}]^T. \quad (22)$$

对于每一组给定的决策向量 $\mathbf{z}_K$ ，从一子级分离状态开始，依次对滑行段和二子级有动力飞行段的动力学方程进行正向积分，并将计算得到的终端残差返回给优化器。该方法仅对控制量进行离散，而状态量仍通过微分方程连续传播，因此称为控制参数化直接打靶法。

由于式 ((21) 式) 中的线性插值系数均位于区间 $[0, 1]$ ，因此只要各控制节点满足

$$-\alpha_{\max} \leq \alpha_j \leq \alpha_{\max}, \quad j = 0, 1, \dots, K-1, \quad (23)$$

则整个时间区间内的夹角约束均能自动成立。

离散后的有限维非线性规划问题可以表示为

$$\begin{aligned} & \min_{\mathbf{z}_K} J(\mathbf{z}_K) = t_{c2} \\ \text{s.t. } & \mathbf{F}(\mathbf{z}_K) = \mathbf{0}, \\ & \mathbf{l} \leq \mathbf{z}_K \leq \mathbf{u}, \end{aligned} \quad (24)$$

其中， $\mathbf{l}$ 和 $\mathbf{u}$ 分别为决策变量的下界向量和上界向量。

5.3.3 SLSQP 求解与网格加密

在第 q 次迭代中, 序列最小二乘二次规划 (SLSQP) 方法通过局部线性化约束, 构造如下二次规划子问题:

$$\begin{aligned} \min_{\mathbf{d}} \quad & \nabla J(\mathbf{z}^{(q)})^T \mathbf{d} + \frac{1}{2} \mathbf{d}^T \mathbf{B}^{(q)} \mathbf{d} \\ \text{s.t.} \quad & \mathbf{F}(\mathbf{z}^{(q)}) + \mathbf{J}_F(\mathbf{z}^{(q)}) \mathbf{d} = \mathbf{0}, \end{aligned} \quad (25)$$

其中, $\mathbf{B}^{(q)}$ 为第 q 次迭代中拉格朗日函数 Hessian 矩阵的拟牛顿近似, $\mathbf{J}_F$ 为终端约束关于决策变量的 Jacobian 矩阵。

每次迭代首先通过有限差分计算目标函数和约束函数对决策变量的敏感度, 随后求解二次规划子问题, 并通过线搜索确定更新步长。重复上述过程, 直至目标函数变化量、迭代步长和约束可行性误差同时达到给定的收敛阈值。

计算初始阶段采用 $K = 9$ 个控制节点, 并分别选取 0 s 和 90 s 作为滑行时间初值。随后将控制网格依次加密至 $K = 17$ 、 $K = 33$ 和 $K = 65$ 个节点, 并将上一层网格得到的最优控制插值到新网格上, 作为下一层优化的热启动初值。优化阶段的最大积分步长设置为 5 s, 最终采用 0.5 s 的积分步长重新积分, 以验证优化结果。

具体求解流程如下: 首先固定控制节点数, 并给定滑行时间、夹角节点和二子级工作时间的初始值; 随后对每一组候选决策向量依次完成滑行段和有动力飞行段积分, 并计算目标函数以及三个缩放后的终端残差; SLSQP 更新决策向量后, 重新进行正向积分。算法收敛后, 进一步执行边界检查、推进剂余量检查和高精度积分复算。

5.3.4 优化结果

表 5 问题三网格加密结果与问题二结果对比

方案	K	滑行时间/s	工作时间/s	剩余推进剂/kg	缩放残差范数
问题二线性俯仰角程序	—	103.631 602	398.048 975	4014.720	—
直接打靶	9	27.066 028	394.649 851	4509.878	3.76×10^{-10}
直接打靶	17	27.231 056	394.649 593	4509.916	4.19×10^{-10}
直接打靶	33	27.383 548	394.649 472	4509.934	6.47×10^{-10}
直接打靶	65	27.383 720	394.649 418	4509.941	7.69×10^{-11}

控制节点加密后, 工作时间与剩余推进剂逐步稳定。65 节点结果较 9 节点仅多保留 0.063 kg 推进剂, 表明控制离散误差已较小。最优夹角在二子级点火初期取 -15° 下界, 随后逐渐进入约束内部, 以较快降低径向速度并完成末段圆化。问题三较问题二多保留约 495.22 kg 推进剂。

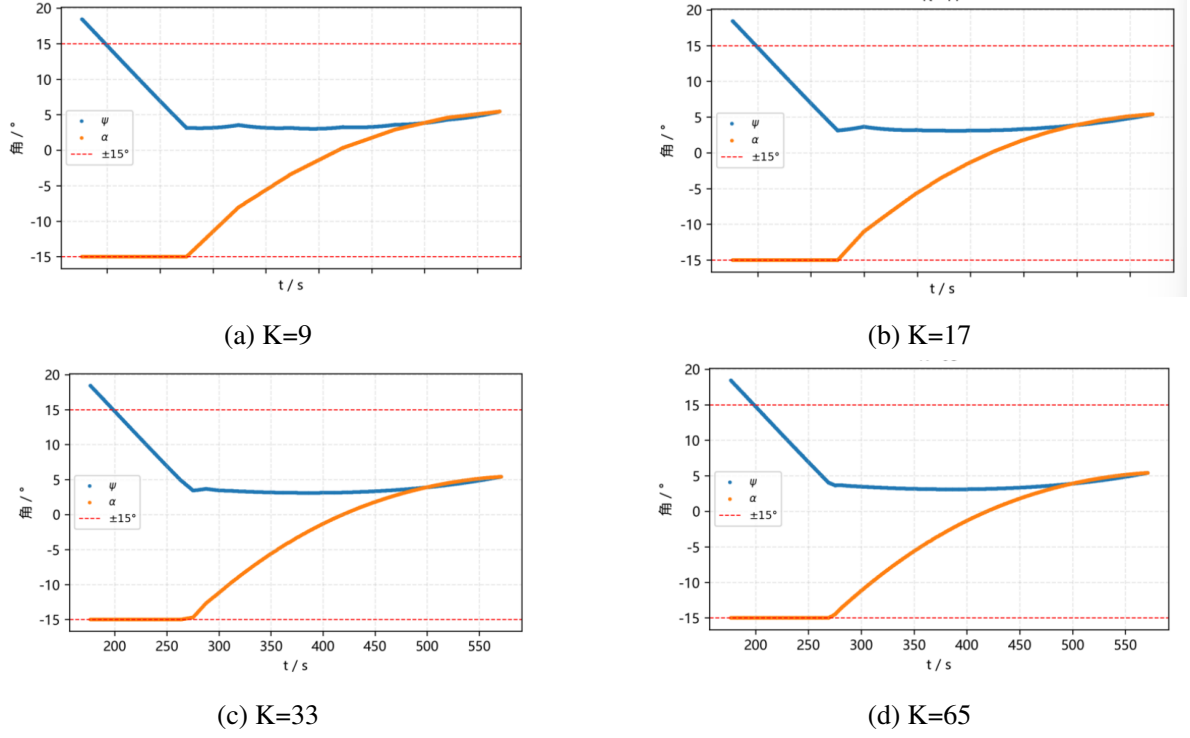


图 4 基准控制策略下全过程状态曲线

5.4 问题四模型建立及求解

5.4.1 可节流动力学与燃料目标

定义二子级节流比 $u(t) = T(t)/F_{\max 2} \in (0.6, 1)$ 。二子级状态方程中推力项统一替换为 $uF_{\max 2}$ ，则状态方程可以表示为

$$\begin{cases} \dot{r} = v_r, \\ \dot{\theta} = \frac{v_t}{r}, \\ \dot{v}_r = \frac{v_t^2}{r} - \frac{\mu}{r^2} + \frac{u(t)F_{\max,2} \sin(\gamma + \alpha)}{m}, \\ \dot{v}_t = -\frac{v_r v_t}{r} + \frac{u(t)F_{\max,2} \cos(\gamma + \alpha)}{m}, \\ \dot{m} = -\frac{u(t)F_{\max,2}}{I_{sp,2}g_0}. \end{cases} \quad (26)$$

令二子级实际工作时间为 b ，则推进剂消耗量由节流积分决定。定义等效满推力时间

$$q = \int_0^b u(s) ds, \quad (27)$$

相应的推进剂消耗量为

$$m_{\text{fuel}} = \frac{F_{\max,2}}{I_{sp,2}g_0} q. \quad (28)$$

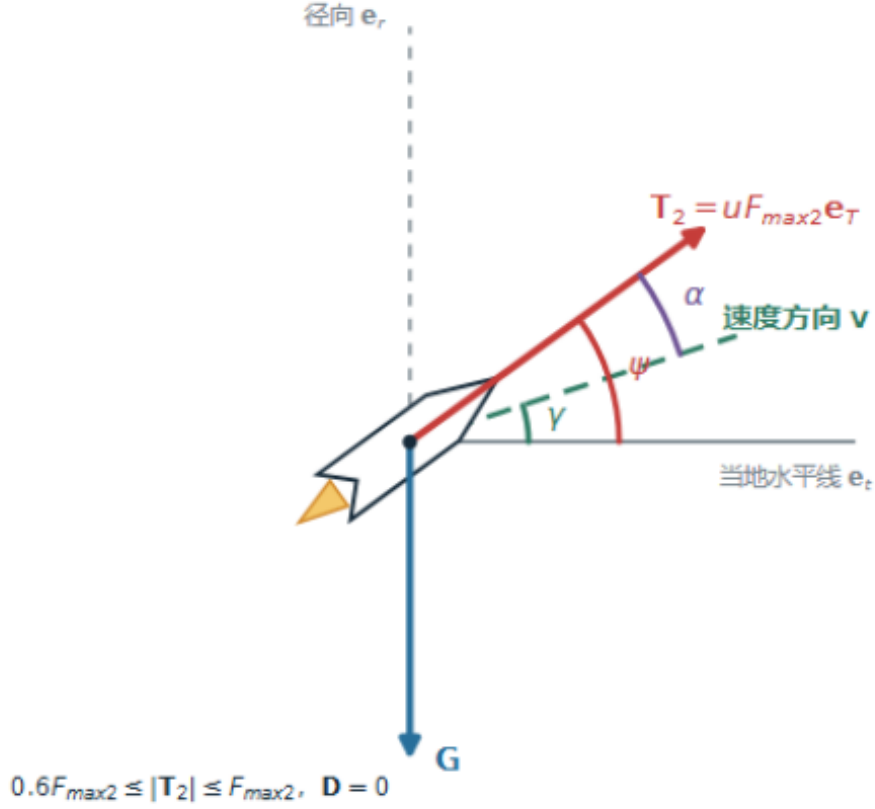


图 5 问题四二子级可节流飞行的受力与控制关系

5.4.2 双控制参数化模型

在相同的 K 个等距控制节点上分别设置推力与速度夹角 α_j 和节流比 u_j ，并在相邻节点之间进行分段线性插值。由于节流比采用分段线性插值，在均匀节点条件下，其平均值可以由梯形公式精确计算：

$$\bar{u} = \frac{\frac{1}{2}u_0 + \sum_{j=1}^{K-2} u_j + \frac{1}{2}u_{K-1}}{K-1}, \quad b = \frac{q}{\bar{u}}, \quad (29)$$

其中， $\bar{u}$ 为平均节流比， q 为等效满推力时间， b 为二子级发动机的实际工作时间。

为改善不同决策变量之间的尺度差异，定义缩放后的决策向量为

$$\mathbf{z} = \left[\frac{\tau}{100} \quad \alpha_0 \quad \cdots \quad \alpha_{K-1} \quad u_0 \quad \cdots \quad u_{K-1} \quad \frac{q}{400} \right]^T. \quad (30)$$

实际工作时间由 $b = q/\bar{u}$ 反算得到，因此任意候选决策向量对应的推进剂消耗量

均与其节流积分保持一致。问题四的有限维优化模型可以表示为

$$\begin{aligned}
& \min_{\mathbf{z}} J(\mathbf{z}) = q \\
& \text{s.t. } \mathbf{F}(\mathbf{z}) = \mathbf{0}, \\
& |\alpha_j| \leq 15^\circ, \quad j = 0, 1, \dots, K-1, \\
& 0.6 \leq u_j \leq 1, \quad j = 0, 1, \dots, K-1, \\
& 0 \leq \tau \leq \tau_{\max}, \\
& 0 < q \leq \frac{m_{p2} I_{\text{sp},2} g_0}{F_{\max,2}}.
\end{aligned} \tag{31}$$

其中，三项等式约束分别对应高度为 400 km 的目标圆轨道半径、零径向速度和目标切向速度。SLSQP 的求解结构与问题三相同，但决策变量的维数增加至

$$\dim(\mathbf{z}) = 2K + 2. \tag{32}$$

对于缩放后的决策变量，目标函数关于最后一个分量的梯度为 400，关于其余分量的梯度均为 0；终端约束的 Jacobian 矩阵采用有限差分方法计算。在 $K = 9$ 个控制节点时，分别采用全推力、80% 推力和 60% 推力作为节流比初值。随后将控制网格依次加密至 $K = 17$ 和 $K = 33$ 个节点，并使用上一层网格的优化结果进行插值，以构造下一层优化的热启动初值。

对于每个收敛的候选解，均重新计算节流积分、剩余推进剂质量和终端状态，并在相同节点数下与问题三的计算结果进行比较。

5.4.3 求解结果与节流判定

不同控制节点数下的网格加密结果如表 表 6 所示。

表 6 问题四网格加密结果

K	滑行时间 (s)	工作时间 (s)	等效满推力时间 (s)	节流范围	剩余推进剂 (kg)	缩放残差范数
9	27.0660	394.650	394.450	1.000 ~ 1.000	4509.878	2.17×10^{-10}
17	27.227	394.650	394.650	1.000 ~ 1.000	4509.916	9.48×10^{-10}
33	27.385	394.649	394.649	1.000 ~ 1.000	4509.933	1.90×10^{-10}

三组 9 节点初值均收敛到同一全程满推力结构，17 和 33 节点加密后节流比仍为 1。由于此时 $q=b$ ，问题四在相同网格上的目标值与问题三一致。该结果表明，在当前动力学、圆轨道终端条件和夹角约束下，降低推力延长飞行时间带来的重力损失超过其可能产生的控制收益，因此允许节流并未进一步减少推进剂消耗。该结论是多初值和网格加密支持的最优候选，不等同于严格的全局最优性证明。

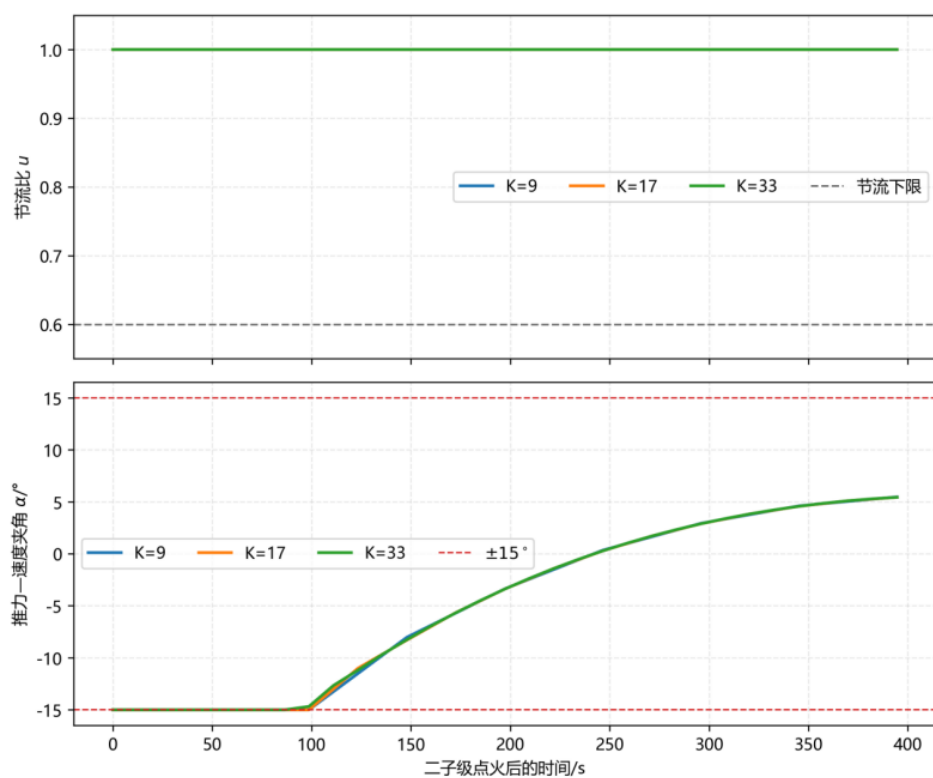


图 6 问题四二子级可节流飞行的受力与控制关系

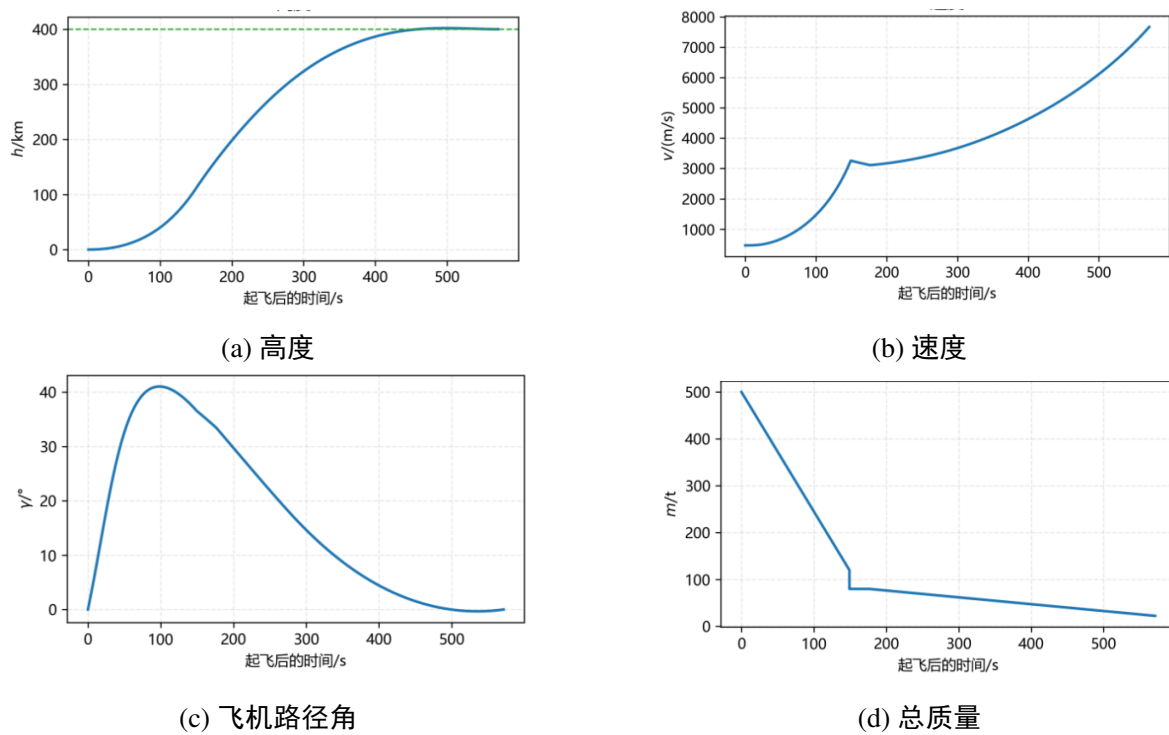


图 7 基准控制策略下全过程状态曲线

5.4.4 数值敏感性检验

由表 7 可知, 在六种积分最大步长下, 剩余推进剂质量均保持为 4509.9335 kg。终端高度的绝对误差均小于 2×10^{-7} m, 缩放残差范数均保持在 10^{-10} 数量级。因此, 计算结果对积分最大步长的变化不敏感, 说明所得结论不依赖于某一特定的最大步长设置。

表 7 问题四中 33 节点解对积分最大步长的敏感性

最大步长 (s)	剩余推进剂 (kg)	高度误差 (m)	径向速度误差 (m · s ⁻¹)	切向速度误差 (m · s ⁻¹)	缩放残差 范数
0.10	4509.934	-1.62×10^{-7}	-3.23×10^{-10}	-9.48×10^{-10}	1.90×10^{-10}
0.25	4509.934	-1.19×10^{-7}	1.44×10^{-11}	1.73×10^{-11}	1.19×10^{-10}
0.50	4509.934	-1.23×10^{-7}	-3.50×10^{-11}	-5.28×10^{-11}	1.23×10^{-10}
1.00	4509.934	-1.19×10^{-7}	-6.47×10^{-12}	-2.36×10^{-11}	1.19×10^{-10}
2.00	4509.934	-1.30×10^{-7}	-2.47×10^{-11}	-3.18×10^{-11}	1.30×10^{-10}
5.00	4509.934	-1.29×10^{-7}	-2.09×10^{-11}	-2.27×10^{-11}	1.29×10^{-10}

6 模型评价与推广

6.1 模型优点

模型在统一的惯性极坐标框架内处理中心引力、地球自转初速度、旋转大气阻力、变质量和级间质量跃变, 避免了不同问题采用不一致状态定义。问题二采用多初值非线性打靶, 问题三、四采用控制参数化、网格加密和独立高精度复算, 数值结果能够相互衔接。问题四重新定义燃料目标, 避免将可节流条件下的工作时间误当作耗油量。

6.1.1 模型不足

本文采用二维质点模型和指数大气, 未描述姿态角速度、发动机摆角速率、节流速率、结构载荷、动压与热流等路径约束; 15° 夹角上限为文献支持的附加建模条件, 并非本题火箭的已知硬件极限。SLSQP 属于局部优化方法, 多初值与网格加密只能增强候选解可信度, 不能给出全局最优性证明。补充的庞特里亚金极小值原理单点打靶在现行径向一切向模型下未达到残差验收标准, 因此未将其旧结果用于本文结论。进一步研究可采用多重打靶或伪谱法, 并加入三维地球模型和工程路径约束。

6.2 模型推广

本文的分阶段动力学与控制参数化框架不依赖某一组固定火箭参数。替换各级质量、推力、比冲和气动参数后, 可用于其他多级运载器的基准轨迹计算与入轨控制设

计；增加目标轨道倾角、分离方位角及三维速度分量后，可推广至非赤道发射和三维轨迹优化。燃料目标还可与动压、过载和热流峰值组成多目标函数，为工程设计提供效率与安全之间的权衡方案。

7 AI 工具使用说明

研究过程中使用生成式人工智能工具辅助资料核对、文字表达和公式排版。动力学方程、约束条件、程序实现与数值结果均以题目资料和团队代码为依据重新计算，并由团队成员人工复核。AI 工具信息与详细使用情况见表 8。

表 8 AI 工具使用说明

项目	说明
工具名称	Kimi
版本或型号	Kimi K3
工具用途	资料核对、语言润色、公式及 Word 排版辅助
结果核验	动力学方程、程序和数值结果均依据团队代码重新计算并人工复核

参考文献

- [1] Hairer E, Nørsett S P, Wanner G. *Solving Ordinary Differential Equations I: Nonstiff Problems*[M]. 2nd ed. Berlin: Springer, 1993.
- [2] Moré J J, Garbow B S, Hillstom K E. *User Guide for MINPACK-1*[R]. Argonne National Laboratory Report ANL-80-74, 1980.
- [3] Kraft D. *A Software Package for Sequential Quadratic Programming*[R]. DFVLR-FB 88-28, 1988.
- [4] Orgeira-Crespo P, et al. Optimization of the Conceptual Design of a Multistage Rocket Launcher[J]. *Aerospace*, 2022, 9(6): 286. DOI: <https://doi.org/10.3390/aerospace9060286>.
- [5] Civek-Coşkun E, Özgören M K. Space Launch Vehicle Design with Simultaneous Optimization of Thrust Profile and Trajectory[C]// *AIAA SPACE and Astronautics Forum and Exposition*. 2017: AIAA 2017-5333. DOI: <https://doi.org/10.2514/6.2017-5333>.
- [6] Betts J T. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*[M]. 2nd ed. Philadelphia: SIAM, 2010.

附录

附录 1: 工具函数的 Python 脚本 (核心部分)

Listing 1: utils.py

```
import numpy as np
from scipy.integrate import solve_ivp
from constant import *
M_AFTER_S1_BURNOUT = M_0 - m_p_1
# 二子级燃尽时总质量
M_AFTER_S2_BURNOUT = m_s_2 + m_payload
DEG = np.pi / 180.0
# 指数大气密度
def air_density(h):
    return rho0 * np.exp(-max(h, 0.0) / h0)
# 旋转大气下的阻力分量: D_r(径向)、D_t(切向), 使用相对速度 v_r, v_t-omega_E*r
def drag_components(h, v_r, v_t, r):
    v_rel_t = v_t - OMEGA_E * r
    V_rel = np.hypot(v_r, v_rel_t)
    k = 0.5 * air_density(h) * C_D * S * V_rel
    return -k * v_r, -k * v_rel_t
# 高度h处圆轨道速度
def v_circular(h):
    return np.sqrt(mu / (R_E + h))
# 一子级俯仰角. 垂直 t_up 秒, 后按 k1(°/s) 线性减小
def psi_stage1(t, t_up, k1_deg_s):
    if t <= t_up:
        return 90.0 * DEG
    return 90.0 * DEG - k1_deg_s * DEG * (t - t_up)
def rhs(t, y, phase, t_up, k1_deg_s, psi_rate_deg_s=0.0, psi0=None,
        t_ignition=0.0, u_throttle=1.0, relative_angle=False):
    """惯性极坐标状态 y=[r, theta, v_r, v_t, m]。
    v_r 径向速度、v_t 沿经线切向(向东)速度; theta 为惯性系极角。
    phase: 's1' 一子级(含垂直+转弯); 'coast' 滑行; 's2' 二子级。
    psi0: 二子级点火初始俯仰角(弧度); 默认沿惯性速度方向。
    psi_rate_deg_s: 二子级俯仰角变化速率 °/s
    u_throttle: 二子级推力节流系数 [0.6, 1.0]
    """
    r, theta, v_r, v_t, m = y
    h = r - R_E
    gamma = np.arctan2(v_r, v_t)
```

```

if phase == 's1':
    psi = psi_stage1(t, t_up, k1_deg_s)
    T = F_max_1
    D_r, D_t = drag_components(h, v_r, v_t, r)
    Isp = I_sp_1
elif phase == 'coast':
    psi = gamma
    T = 0.0
    D_r = 0.0
    D_t = 0.0
    Isp = I_sp_1
else:
    if psi0 is None:
        psi0 = 0.0 if relative_angle else gamma
    psi = psi0 + psi_rate_deg_s * DEG * (t - t_ignition)
    if relative_angle:
        psi += gamma
    T = F_max_2 * u_throttle
    D_r = 0.0
    D_t = 0.0
    Isp = I_sp_2
g_term = mu / (r * r)
dr = v_r
dtheta = v_t / r
dv_r = v_t * v_t / r - g_term + (T * np.sin(psi) + D_r) / m
dv_t = -v_r * v_t / r + (T * np.cos(psi) + D_t) / m
dm = -T / (Isp * g0)
return np.array([dr, dtheta, dv_r, dv_t, dm])
def event_s1_burnout(t, y):
    return y[4] - M_AFTER_S1_BURNOUT
def event_s2_burnout(t, y):
    return y[4] - M_AFTER_S2_BURNOUT
event_s1_burnout.terminal = True
event_s2_burnout.terminal = True
event_s1_burnout.direction = -1
event_s2_burnout.direction = -1
# 统一积分器:DOP853,高精度小步长
def integrate(t_span, y0, phase, event=None, args=None, *, max_step
=0.5):
    a = args or {}
    kwargs = dict(rtol=1e-9, atol=1e-12, max_step=max_step, method='

```

```

    DOP853')
    if event is not None:
        kwargs['events'] = event
    return solve_ivp(lambda t, y: rhs(t, y, phase, **a), t_span, y0, **
        kwargs)
# 启发式搜索用粗积分:精度牺牲换速度,精修阶段再走高精度版
def integrate_cheap(t_span, y0, phase, event=None, args=None):
    a = args or {}
    kwargs = dict(rtol=1e-6, atol=1e-9, max_step=2.0, method='RK45')
    if event is not None:
        kwargs['events'] = event
    return solve_ivp(lambda t, y: rhs(t, y, phase, **a), t_span, y0, **
        kwargs)
# 二子级点火的右端:接受 psi_program(t) 作为角度程序
def rhs_s2(t, y, psi_t, t_ignition, u_throttle=1.0):
    r, theta, v_r, v_t, m = y
    gamma = np.arctan2(v_r, v_t)
    psi = psi_t(t - t_ignition)
    T = F_max_2 * u_throttle
    g_term = mu / (r * r)
    dr = v_r
    dtheta = v_t / r
    dv_r = v_t * v_t / r - g_term + T * np.sin(psi) / m
    dv_t = -v_r * v_t / r + T * np.cos(psi) / m
    dm = -T / (I_sp_2 * g0)
    return np.array([dr, dtheta, dv_r, dv_t, dm])
# 通用 forward: 滑行 + 二子级分段线性 psi 积分
def forward_coast_s2(y_after_sep, tau, psi_nodes, psi_times, t_c2,
    t_up, k1_deg_s, u_throttle=1.0, integrator=None, initial_angle=None,
    relative_angle=False):
    if integrator is None:
        integrator = integrate
    sol_c = integrator((0.0, tau), y_after_sep, 'coast', args=dict(t_up=
        t_up, k1_deg_s=k1_deg_s))
    y_ign = sol_c.y[:, -1]
    if not sol_c.success:
        raise RuntimeError(sol_c.message)
    # 点火处惯性速度方向 gamma = atan2(v_r, v_t)
    ign_gamma = np.arctan2(y_ign[2], y_ign[3])
    if initial_angle is None:
        psi0_fixed = ign_gamma if not relative_angle else 0.0

```

```

else:
    psi0_fixed = float(initial_angle)
    pts_t = [0.0]
    pts_psi = [psi0_fixed]
    last_t = 0.0
    for tk, pk in zip(psi_times, psi_nodes):
        if tk <= last_t + 1e-6:
            continue
        if tk > t_c2:
            tk = t_c2
        pts_t.append(float(tk))
        pts_psi.append(float(pk))
        last_t = tk
    if pts_t[-1] < t_c2 - 1e-6:
        pts_t.append(t_c2)
        pts_psi.append(pts_psi[-1])
    y_cur = y_ign.copy()
    cur_t = 0.0
    ts_all, ys_all = [], []
    for i in range(len(pts_t) - 1):
        t_a, t_b = pts_t[i], pts_t[i + 1]
        psi_a, psi_b = pts_psi[i], pts_psi[i + 1]
        seg = t_b - t_a
        if seg <= 1e-9:
            continue
        psi_rate_deg = (psi_b - psi_a) / seg / DEG
        sol_seg = integrator((cur_t, cur_t + seg), y_cur, 's2', args=dict(
            t_up=t_up, k1_deg_s=k1_deg_s, psi_rate_deg_s=psi_rate_deg,
            t_ignition=cur_t, psi0=psi_a, u_throttle=u_throttle,
            relative_angle=relative_angle))
        if not sol_seg.success:
            raise RuntimeError(sol_seg.message)
        ts_all.append(sol_seg.t)
        ys_all.append(sol_seg.y)
        y_cur = sol_seg.y[:, -1].copy()
        cur_t += seg
    ts = np.concatenate(ts_all)
    ys = np.concatenate(ys_all, axis=1)
    class _Sol:
    pass
    sol_s2 = _Sol()

```

```

sol_s2.t = ts
sol_s2.y = ys
y_end = y_cur
return y_ign, sol_c, sol_s2, y_end
# 终止条件:三个终端偏差,惯性极坐标下对应圆轨道 h=H, v_r=0, v_t=v_circ。
def orbit_residual3(y_end):
    r, _, v_r, v_t, _ = y_end
    h = r - R_E
    return np.array([h - H_orbit, v_r, v_t - v_circular(H_orbit)])
def orbit_residual3_scaled(y_end):
return orbit_residual3(y_end) / np.array([1e3, 10.0, 10.0])
# 燃尽最大时长(二子级)
def t_b2_max():
    return m_p_2 * I_sp_2 * g0 / F_max_2

```

附录 2: 问题一求解的 Python 脚本 (核心部分)

Listing 2: probl.py

```

import os
import numpy as np
from constant import M_0, m_s_1, t_up, k1_baseline, tau_baseline, R_E,
    OMEGA_E
from utils import event_s1_burnout, event_s2_burnout, plot_curves, DEG
, integrate
os.makedirs('../out/probl', exist_ok=True)
def main():
    y0 = np.array([R_E, 0.0, 0.0, OMEGA_E * R_E, M_0])
    sol1 = integrate((0.0, 1e4), y0, 's1', event=event_s1_burnout, args=
        dict(t_up=t_up, k1_deg_s=k1_baseline))
    t1_end = sol1.t[-1]
    y1_end = sol1.y[:, -1]
    gamma1 = np.degrees(np.arctan2(y1_end[2], y1_end[3]))
    y_after_sep = y1_end.copy()
    y_after_sep[4] -= m_s_1
    sol2 = integrate((t1_end, t1_end + tau_baseline), y_after_sep, '
        coast', args=dict(t_up=t_up, k1_deg_s=k1_baseline))
    t2_end = sol2.t[-1]
    y2_end = sol2.y[:, -1]
    gamma2 = np.degrees(np.arctan2(y2_end[2], y2_end[3]))
    sol3 = integrate((t2_end, t2_end + 1e4), y2_end, 's2', event=

```



```

    event_s2_burnout, args=dict(t_up=t_up, k1_deg_s=k1_baseline,
                                t_ignition=t2_end))
t3_end = sol3.t[-1]
y3_end = sol3.y[:, -1]
gamma3 = np.degrees(np.arctan2(y3_end[2], y3_end[3]))
ts = np.concatenate([sol1.t, sol2.t[1:], sol3.t[1:]])
ys = np.concatenate([sol1.y, sol2.y[:, 1:], sol3.y[:, 1:]], axis=1)
seg_idx = np.concatenate([np.zeros(len(sol1.t)), np.ones(len(sol2.t)
    - 1), np.full(len(sol3.t) - 1, 2) ]).astype(int)
if __name__ == '__main__':
    main()

```

附录 3： 问题二求解的 Python 脚本（核心部分）

Listing 3: prob2.py

```

import os
import numpy as np
from scipy.optimize import fsolve
from constant import M_0, m_s_1, t_up, k1_baseline, H_orbit, R_E
from utils import event_s1_burnout, plot_curves, DEG, v_circular,
    M_AFTER_S2_BURNOUT, integrate
os.makedirs('../out/prob2', exist_ok=True)
def _forward(y_after_sep, tau, k2_deg_s, t_c2):
    t_split = 0.0 # 本地时间原点
    t_ign = tau
    sol_c = integrate((t_split, t_split + tau), y_after_sep, 'coast',
        args=dict(t_up=t_up, k1_deg_s=k1_baseline))
    y_ign = sol_c.y[:, -1]
    sol_s2 = integrate((t_ign, t_ign + t_c2), y_ign, 's2', args=dict(
        t_up=t_up, k1_deg_s=k1_baseline,
        psi_rate_deg_s=k2_deg_s,
        t_ignition=t_ign, psi0=float(y_ign[3])))
    y_end = sol_s2.y[:, -1]
    m_remain = y_end[4] - M_AFTER_S2_BURNOUT
    return y_end, m_remain
def _residual(x, y_after_sep):
    tau, k2, t_c2 = x
    if tau < 0 or t_c2 <= 0:
        return np.array([1e6, 1e6, 1e6])
    try:

```

```

    y_end, m_rem = _forward(y_after_sep, tau, k2, t_c2)
except Exception:
    return np.array([1e6, 1e6, 1e6])
r, _, v, gamma, _ = y_end
h = r - R_E
# 剩余燃料为负：解无效
if m_rem < -0.0:
    err = 1e3 - m_rem
    return np.array([err, err, err])
return np.array([h - H_orbit, v - v_circular(H_orbit), gamma])
def multi_start_check(y_after_sep, path='../out/prob2/test.csv'):
    xs, iers = [], []
    grid = [(t0, k0, c0)
    for t0 in (0.0, 60.0, 150.0)
    for k0 in (-0.5, 0.0, 0.5)
    for c0 in (200.0, 300.0, 420.0)]
    for idx, x0 in enumerate(grid):
        x, _, ier, _ = fsolve(_residual, list(x0), args=(y_after_sep,),
            full_output=True, xtol=1e-10)
        xs.append(x)
        iers.append(ier)
    conv = np.array(xs)[np.array(iers) == 1]
    unique_solutions = {}
    for p in conv:
        key = tuple(np.round(p, 1))
        unique_solutions.setdefault(key, p)
def main():
    y0 = np.array([R_E, 0.0, 0.0, 90.0 * DEG, M_0])
    sol1 = integrate((0.0, 1e4), y0, 's1', event=event_s1_burnout,
        args=dict(t_up=t_up, k1_deg_s=k1_baseline))
    t1_end = sol1.t[-1]
    y_after_sep = sol1.y[:, -1].copy()
    y_after_sep[4] -= m_s_1
    # 用 prob1 结果作为初值便于收敛
    x0 = np.array([60.0, 0.0, 420.0])
    sol_root = fsolve(_residual, x0, args=(y_after_sep,), full_output=
        True, xtol=1e-10)
    x_star, info, ier, _ = sol_root
    tau, k2, t_c2 = x_star
    t_ign = t1_end + tau
    y_ign = sol_c.y[:, -1]

```

```

sol_s2 = integrate((t_ign, t_ign + t_c2), y_ign, 's2',
args=dict(t_up=t_up, k1_deg_s=k1_baseline,
psi_rate_deg_s=k2,
t_ignition=t_ign, psi0=float(y_ign[3])))
y_end = sol_s2.y[:, -1]
m_remain = y_end[4] - M_AFTER_S2_BURNOUT
ts = np.concatenate([sol1.t, sol_c.t[1:], sol_s2.t[1:]])
ys = np.concatenate([sol1.y, sol_c.y[:, 1:], sol_s2.y[:, 1:]], axis
=1)
seg_idx = np.concatenate([
np.zeros(len(sol1.t)),
np.ones(len(sol_c.t) - 1),
np.full(len(sol_s2.t) - 1, 2),
]).astype(int)
t_bnd = [t1_end, t_ign]
plot_curves(ts, ys, t_boundaries=t_bnd, save_name='../out/prob2/plot
')
# 唯一根检验
multi_start_check(y_after_sep)
if __name__ == '__main__':
    main()

```

附录 4： 问题三求解的 Python 脚本（核心部分）

Listing 4: prob3_SLSQP.py

```

from pathlib import Path
from functools import partial
from time import perf_counter
import numpy as np
from scipy.optimize import minimize
from constant import M_0, m_s_1, t_up, k1_baseline, R_E, OMEGA_E,
alpha_max_deg
from utils import event_s1_burnout, DEG, integrate, forward_coast_s2,
orbit_residual3_scaled, t_b2_max, M_AFTER_S2_BURNOUT
K_LIST = (9, 17, 33, 65)
T_B2_MAX = t_b2_max()
TAU_MAX = 600.0
ALPHA_MAX = alpha_max_deg * DEG
OUT_DIR = Path(__file__).resolve().parents[1] / 'out' / 'prob3'
integrate_search = partial(integrate, max_step=5.0)

```

```

def decode(x, K):
    tau, t_c2 = float(x[0]), float(x[-1])
    alpha_nodes = np.asarray(x[1:-1])
    return alpha_nodes, np.linspace(0.0, t_c2, K), tau, t_c2

def warm_start(previous, K):
    return np.r_[previous[0], np.interp(np.linspace(0., 1., K), np.
        linspace(0., 1., len(previous)-2), previous[1:-1]), previous[-1]]

def _safe_forward(y_after_sep, x, K, cheap=True):
    x = np.asarray(x)
    if (x.shape != (K + 2,) or not np.all(np.isfinite(x)) or not 0 <= x
        [0] <= TAU_MAX or not 0 < x[-1] <= T_B2_MAX or np.any(np.abs(x
        [1:-1]) > ALPHA_MAX + 1e-12)):
        return None
    nodes, times, tau, burn = decode(x, K)
    try:
        return forward_coast_s2(y_after_sep, tau, nodes[1:], times[1:],
            burn, t_up, k1_baseline, integrator=integrate_search if cheap
            else integrate, initial_angle=nodes[0], relative_angle=True)
    Except :
        return None

def neg_mremain(x, y_after_sep, K):
    out = _safe_forward(y_after_sep, x, K, cheap=False)
    return 1e6 if out is None else -(out[3][4] - M_AFTER_S2_BURNOUT)

def residual_3(x, y_after_sep, K, cheap=False):
    out = _safe_forward(y_after_sep, x, K, cheap=cheap)
    return np.full(3, 1e6) if out is None else orbit_residual3_scaled(
        out[3])

def solve_one(K, y_after_sep, previous=None):
    bounds = [(0.0, TAU_MAX)] + [(-ALPHA_MAX, ALPHA_MAX)] * K + [(20.,
        T_B2_MAX)]
    candidates = []
    seeds = ([warm_start(previous, K)] if previous is not None else [np.
        r_[tau0, np.linspace(-ALPHA_MAX, 3*DEG, K), 413.] for tau0 in
        (0., 90.)])
    for x0 in seeds:
        start = perf_counter()
        for cheap in (True, False):
            fit = minimize(lambda x: x[-1], x0, method='SLSQP', bounds=bounds
                , constraints=[{'type': 'eq', 'fun': lambda x: residual_3(x,
                y_after_sep, K, cheap)}], options={'ftol': 1e-9, 'maxiter':
                300})

```

```

    error = np.linalg.norm(residual_3(fit.x, y_after_sep, K, cheap=
        False))
    if fit.success and error < 1e-5:
        break
    x0 = fit.x
    error = np.linalg.norm(residual_3(fit.x, y_after_sep, K))
    if fit.success and error < 1e-5:
        candidates.append(fit)
if not candidates:
    if previous is not None:
        return solve_one(K, y_after_sep)
    return None
return min(candidates, key=lambda fit: fit.fun)
def main():
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    y0 = np.array([R_E, 0.0, 0.0, OMEGA_E * R_E, M_0])
    sol1 = integrate((0.0, 1e4), y0, 's1', event=event_s1_burnout,
        args=dict(t_up=t_up, k1_deg_s=k1_baseline))
    y_after_sep = sol1.y[:, -1].copy()
    y_after_sep[4] -= m_s_1
    t1 = sol1.t[-1]
    psi1 = np.where(sol1.t <= t_up, 90.0, 90.0-k1_baseline*(sol1.t-t_up)
        )
    summary_rows = []
    previous = None
    for K in K_LIST:
        start = perf_counter()
        best = solve_one(K, y_after_sep, previous=previous)
        elapsed = perf_counter()-start
        if best is None:
            continue
        x_star = best.x
        nodes, times, tau, burn = decode(x_star, K)
        out = _safe_forward(y_after_sep, x_star, K, cheap=False)
        _, sol_c, sol_s2, y_end = out
        alpha = np.interp(sol_s2.t, times, nodes)
        gamma = np.arctan2(sol_s2.y[2], sol_s2.y[3])
        psi = gamma + alpha
        max_alpha = float(np.max(np.abs(alpha)) / DEG)
        error = np.linalg.norm(orbit_residual3_scaled(y_end))
        remain = y_end[4] - M_AFTER_S2_BURNOUT

```

```

ts = np.r_[sol1.t, sol_c.t[1:]+t1, sol_s2.t+t1+tau]
ys = np.concatenate([sol1.y, sol_c.y[:, 1:], sol_s2.y], axis=1)
seg = np.r_[np.zeros(len(sol1.t)), np.ones(len(sol_c.t)-1),
np.full(len(sol_s2.t), 2)]
psi_all = np.r_[psi1, np.full(len(sol_c.t)-1, np.nan), psi/DEG]
alpha_all = np.r_[np.full(len(sol1.t)+len(sol_c.t)-1, np.nan),
alpha/DEG]
summary_rows.append([tau, burn, remain, alpha_max_deg, max_alpha, K
, error, elapsed])
previous = x_star.copy()
if __name__ == '__main__':
    main()

```